

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Работа с базой данных MySQL

Для пентестера очень важно уметь обращаться с базами данных. Если таких навыков у вас нет, то рекомендуется ознакомиться с серией статей на форуме.

[Часть 1](#) [Часть 2](#) [Часть 3](#) [Часть 4](#) [Часть 5](#) [Часть 6](#)

Подключение к базе данных

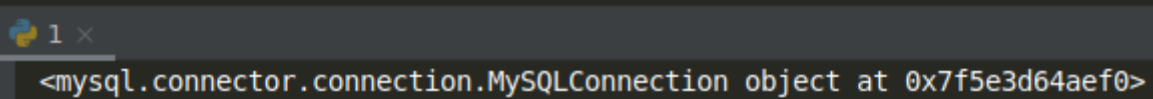
Для работы с базой данных сначала установим модуль `mysql-connector-python` в `pycharm` или через терминал `pip3 install mysql-connector-python`.

Импортируем модуль, и проверим соединение с базой данных. для подключения к базе данных MySQL мы используем функцию `connect()`, которая принимает следующие параметры: хост, база данных, пользователь и пароль. Функция `connect()` устанавливает соединение с базой данных, и возвращает объект `MySQLConnection`.

В Kali Linux по умолчанию пользователь `root` и пароль отсутствует. Для проверки в коде можно писать `database='mysql'`, или не указывать базу, тогда обращение будет к `information_schema`. При некорректных данных выпадут ошибки.

```
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    passwd=""
)
print(mydb)
```

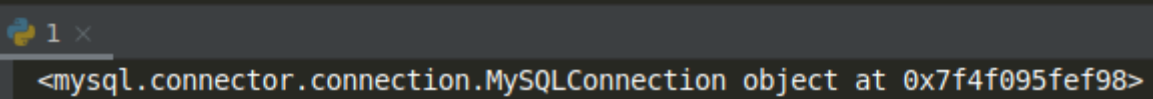


The terminal output shows the result of the `print(mydb)` statement: `<mysql.connector.connection.MySQLConnection object at 0x7f5e3d64aef0>`. This indicates that the connection to the MySQL database was successful.

Мы видим, что создан объект, значит соединение прошло успешно. Чтобы не писать столь длинную команду, сделаем обращение к модулю более коротким.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
print(mydb)
```



The terminal output shows the result of the `print(mydb)` statement: `<mysql.connector.connection.MySQLConnection object at 0x7f4f095fef98>`. This indicates that the connection to the MySQL database was successful using the shorter module name `sql`.

Создание базы данных

Теперь создадим новую базу `pentest`. Если этот код выполнится без ошибок, значит база успешно создана. Метод `cursor()` делает запросы и получает их результаты.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("CREATE DATABASE pentest")
```

Теперь проверим что база действительно создана, и выведем все базы.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SHOW DATABASES")
for db in mycursor:
    print(db)
```

```
1 x
('bwAPP',)
('buratino',)
('golden_key',)
('information_schema',)
('modx',)
('my_db',)
('mysql',)
('pentest',)
('performance_schema',)
('phpmyadmin',)
```

Все вышеприведённые примеры стоит писать с командой `mydb.close()` для того чтобы соединение закрывалось после выполнения нужной операции. Мы можем использовать весь арсенал SQL запросов. Например, узнаем версию базы в следующем примере.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SELECT VERSION()")
for db in mycursor:
    print("Database version : %s " % db)
mydb.close()
```

```
1 x
Database version : 10.3.15-MariaDB-1
```

Также можно делать несколько запросов одновременно, перечислив их через запятую.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SELECT USER(), @@PORT")
for db in mycursor:
    print("Database user : %s\nDatabase port : %s" % db)
mydb.close()
```

```
1 x
Database user : root@localhost
Database port : 3306
```

Во всех примерах выше использовался цикл `for`, но если нам нужно получить всего одно значение, то использование цикла нецелесообразно. Следующий способ получить вывод без цикла, это метод **`fetchone()`**.

Метод **`fetchone()`** —делает выбор одной строки набора результатов запроса. Набор результатов – это возвращаемый объект, когда объект курсора используется для запроса.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SELECT @@datadir")
result = mycursor.fetchone()
print("Database dir : %s " % result)
mydb.close()
```

```
1 x
Database dir : /var/lib/mysql/
```

Соответственно при запросе вывода баз данных мы получим только 1 результат в выводе.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("show databases")
result = mycursor.fetchone()
print("Database: %s " % result)
mydb.close()
```

```
1 x
Database: bwAPP
```

Метод **fetchmany(n)** – извлекает указанное количество строк из набора результатов, где n целое число. Если число будет больше количества строк, то выведутся все результаты в виде списка.


```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("show databases")
result = mycursor.fetchmany(3)
print("Databases : %s " % result)
mydb.close()
```

1 x

Databases : [('bwAPP',), ('buratino',), ('golden_key',)]

Метод **fetchall()** – извлекает все строки в результирующем наборе. Если некоторые строки уже извлечены из набора результатов, то он извлекает оставшиеся строки из набора результатов в виде списка.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("show databases")
result = mycursor.fetchall()
print("Databases:\n%s " % result)
```

1 x

Databases:
[('bwAPP',), ('buratino',), ('golden_key',), ('information_schema',), ('modx',)]

Создание таблицы базы данных

При создании таблицы вы должны создать столбец с уникальным ключом для каждой записи. Это можно сделать, определив **ПЕРВИЧНЫЙ КЛЮЧ**.

Мы используем выражение «**INT AUTO_INCREMENT PRIMARY KEY**», которое вставляет уникальный номер для каждой записи. Начиная с 1, и увеличивается на единицу для каждой записи.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY,\
                login VARCHAR(255),\
                pass VARCHAR(255))")
mydb.close()
```

Если ошибок нет, то таблица создана. Проверить можно выводом таблицы базы как через программу в пайтон, так и в терминале.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SHOW TABLES")
result = mycursor.fetchone()
print("table_name: %s" % result)
mydb.close()
```

```
1 x
table_name: users
```

```
MariaDB [pentest]> show tables;
+-----+
| Tables_in_pentest |
+-----+
| users              |
+-----+
1 row in set (0.000 sec)
```

Вставка данных в таблицу

Чтобы заполнить таблицу в MySQL, используйте оператор **INSERT INTO**. А метод **commit()** отвечает за сохранение изменений. Атрибут **rowcount** возвращает количество обновленных(или созданных) строк. В нашем случае, одна строка была создана.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
ins_tab = "INSERT INTO users (login, pass) VALUES (%s, %s)"
val = ("Admin", "Super-Password")
mycursor.execute(ins_tab, val)

mydb.commit()
print(mycursor.rowcount, "Record inserted")
mydb.close()
```

1 x
1 Record inserted

Чтобы вставить несколько строк в таблицу, используйте метод [executemany\(\)](#).

Второй параметр executemany() метода - это список кортежей, содержащий данные, которые вы хотите вставить.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
ins_tab = "INSERT INTO users (login, pass) VALUES (%s, %s)"
val = [
    ("dodik", "12345"),
    ("man", "Batman_$00$"),
    ("Any", "pinkgirl")]
mycursor.executemany(ins_tab, val)

mydb.commit()
print(mycursor.rowcount, "Record inserted")
mydb.close()
```


Вывод данных из таблицы

Теперь выведем все данные из таблицы. Для этого воспользуемся оператором звёздочка*.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SELECT * from users")
for db in mycursor:
    print(db)
mydb.close()
```

```
1 x
(1, 'Admin', 'Super-Password')
(2, 'dodik', '12345')
(3, 'man', 'Batman_$00$')
(4, 'Anya', 'pinkgirl')
```

Если же нас интересует определённая запись, то её можно вывести, например, по id или любому другому значению. Для этого применим оператор WHERE.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
mycursor.execute("SELECT * from users WHERE id=3")
result = mycursor.fetchone()
print(result)
```

```
1 x
(3, 'man', 'Batman_$00$')
```

Удаление данных

Чтобы удалить запись из существующей таблицы, воспользуемся оператором **DELETE**. Удалить можно как отдельную запись, так и всю таблицу или базу.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
sql_del = "DELETE FROM users WHERE login='dodik'"
mycursor.execute(sql_del)
mydb.commit()
print(mycursor.rowcount, "record(s) deleted")
mydb.close()
```

```
1 x
1 record(s) deleted
```

Обновление данных

При необходимости можно обновить данные. Например, админ решил сменить пароль на более надёжный.

```
import mysql.connector as sql

mydb = sql.connect(
    host="localhost",
    database="pentest",
    user="root",
    passwd=""
)
mycursor = mydb.cursor()
sql_del = "UPDATE users SET pass = 'Fjh5@jlhs5K852@pd4' WHERE id=1"
mycursor.execute(sql_del)
mydb.commit()
print(mycursor.rowcount, "record(s) modified")
mydb.close()
```

```
1 x
1 record(s) modified
```

Результат наших двух манипуляций – одна запись удалена и одна обновлена.

```
MariaDB [pentest]> SELECT * from users;  
+-----+-----+-----+  
| id | login | pass |  
+-----+-----+-----+  
| 1 | Admin | Fjh5@jlhs5K852@pd4 |  
| 3 | man | Batman_$00$ |  
| 4 | Anya | pinkgirl |  
+-----+-----+-----+  
3 rows in set (0.000 sec)
```